

TECHNICAL ARCHITECTURE

AI Liability Risk Assessment Agent

Phased Development Plan with Claude Code
From Zero to Acquisition-Ready in 6 Sprints

Stack: Python FastAPI • PostgreSQL • React • Claude API • AWS

Development: Claude Code with Sub-Agents

April 2026 • 2-Person Team

CONFIDENTIAL

1. System Architecture Overview

This document provides a step-by-step technical blueprint for building the AI Liability Risk Assessment Agent. Every section includes exact technology choices, database schemas, API contracts, and — critically — the Claude Code prompts and sub-agent configurations you’ll use to build each component.

1.1 High-Level Architecture

The system follows a clean layered architecture with five distinct services:

Layer	Service	Technology	Purpose
Ingestion	Document Processor	Python + Unstructured.io	Parse PDFs/DOCX, classify document types, chunk and store
Extraction	Governance Extractor	FastAPI + Claude API	LLM-based structured data extraction from governance docs
Enrichment	External Signals	Python scrapers + APIs	AI litigation data, regulatory actions, media sentiment
Scoring	Risk Engine	Python (rules-based)	Deterministic, configurable scoring across 8 governance dimensions
Presentation	Dashboard + Reports	React + PDF generation	Underwriter-facing UI, report generation, file management

1.2 Technology Stack — Final Decisions

Component	Choice	Why This Over Alternatives
Language	Python 3.12+	Best PDF/NLP ecosystem; FastAPI is async-native; Claude SDK is Python-first
API Framework	FastAPI	Async, auto-generated OpenAPI docs, Pydantic validation, dependency injection
Database	PostgreSQL 16 (via Supabase)	JSONB for flexible governance data, pgvector for embeddings, Row Level Security, Supabase gives you auth + storage + realtime for free
Vector Store	pgvector (in PostgreSQL)	No separate service to manage; good enough at our scale (<100K vectors); keeps infra simple
ORM	SQLAlchemy 2.0 + Alembic	Async support; Alembic for migrations; well-understood by Claude Code
LLM	Claude API (Anthropic)	Best document reasoning; 200K context; structured output via tool_use; reliable for extraction

Document Processing	Unstructured.io (open source)	Handles messy insurance PDFs; table extraction; image OCR; self-hostable
Frontend	React 18 + TypeScript + Tailwind + shadcn/ui	Fast to build with Claude Code; shadcn components are copy-paste; Tailwind for rapid styling
State Management	TanStack Query (React Query)	Server-state focused; caching; mutations; perfect for API-driven dashboards
Hosting	AWS (ECS Fargate + RDS + S3 + CloudFront)	Fargate = serverless containers, no EC2 management; RDS for managed Postgres; S3 for documents
CI/CD	GitHub Actions	Free for private repos; Claude Code integrates natively with GitHub
Auth	Supabase Auth (or Clerk)	JWT-based; social login; row-level security in Postgres; zero custom auth code
PDF Reports	WeasyPrint	HTML-to-PDF; supports CSS; produces professional insurance reports
Monitoring	Sentry + PostHog	Sentry for errors; PostHog for product analytics; both have generous free tiers

WHY POSTGRESQL OVER SEPARATE VECTOR DB

At our scale (hundreds, not millions of documents), pgvector inside PostgreSQL eliminates an entire service from the architecture. You get vector similarity search, relational data, and JSONB flexibility in one database. When you hit scale limits (unlikely before acquisition), migrating to Pinecone is straightforward since the embedding format is identical.

2. Database Schema

The schema is designed around the core workflow: an Organization submits Documents, which produce Extractions, which feed into a Score. Everything is audit-trailed.

2.1 Core Tables

```
-- organizations: the companies being assessed
CREATE TABLE organizations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  industry     TEXT,
  employee_count INTEGER,
  ai_systems_count INTEGER,
  metadata     JSONB DEFAULT '{}',
```

```

    created_at      TIMESTAMPTZ DEFAULT now(),
    updated_at      TIMESTAMPTZ DEFAULT now()
);

-- assessments: a single risk assessment request
CREATE TABLE assessments (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id),
    status          TEXT DEFAULT 'pending'
                  CHECK (status IN ('pending', 'processing',
                                   'extraction_complete', 'scoring', 'complete', 'failed')),
    requested_by    UUID REFERENCES auth.users(id),
    overall_score    NUMERIC(5,2),
    risk_tier        TEXT CHECK (risk_tier IN ('low', 'medium', 'high', 'critical')),
    report_url       TEXT,
    config           JSONB DEFAULT '{}', -- carrier-specific scoring weights
    created_at      TIMESTAMPTZ DEFAULT now(),
    completed_at     TIMESTAMPTZ
);

```

```

-- documents: uploaded governance documentation
CREATE TABLE documents (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    assessment_id    UUID REFERENCES assessments(id) ON DELETE CASCADE,
    filename         TEXT NOT NULL,
    file_url         TEXT NOT NULL,          -- S3 path
    file_size_bytes  INTEGER,
    mime_type        TEXT,
    doc_type         TEXT CHECK (doc_type IN (
        'model_card', 'model_inventory', 'bias_audit', 'hitl_policy',
        'incident_response', 'data_governance', 'dpia', 'deployment_arch',
        'third_party_risk', 'regulatory_compliance', 'other', 'unknown'
    )),
    classification_confidence NUMERIC(3,2),
    page_count       INTEGER,
    processing_status TEXT DEFAULT 'uploaded',
    raw_text         TEXT,                  -- full extracted text
    chunks          JSONB,                 -- chunked text with metadata
    created_at      TIMESTAMPTZ DEFAULT now()
);

```

```

-- extractions: structured governance data pulled from documents
CREATE TABLE extractions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id     UUID REFERENCES documents(id) ON DELETE CASCADE,
    assessment_id    UUID REFERENCES assessments(id) ON DELETE CASCADE,
    dimension        TEXT NOT NULL CHECK (dimension IN (
        'model_inventory', 'human_oversight', 'bias_fairness',
        'data_governance', 'incident_response', 'monitoring_drift',

```

```

    'regulatory_compliance','third_party_risk'
  )),
  findings          JSONB NOT NULL,          -- structured extraction output
  evidence          JSONB NOT NULL,          -- source page refs, quotes
  confidence        NUMERIC(3,2),
  llm_model         TEXT,                    -- which Claude model was used
  llm_prompt_hash   TEXT,                    -- for reproducibility
  token_count       INTEGER,
  created_at        TIMESTAMPTZ DEFAULT now()
);

```

```

-- dimension_scores: per-dimension scoring results
CREATE TABLE dimension_scores (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  assessment_id     UUID REFERENCES assessments(id) ON DELETE CASCADE,
  dimension         TEXT NOT NULL,
  score            NUMERIC(5,2) NOT NULL, -- 0-100
  max_score        NUMERIC(5,2) DEFAULT 100,
  weight           NUMERIC(3,2),          -- configurable per carrier
  flags            JSONB DEFAULT '[]',    -- red/amber/green items
  evidence_refs    JSONB DEFAULT '[]',    -- links to extraction evidence
  created_at       TIMESTAMPTZ DEFAULT now()
);

-- external_signals: litigation, regulatory, media data
CREATE TABLE external_signals (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  assessment_id     UUID REFERENCES assessments(id) ON DELETE CASCADE,
  signal_type       TEXT CHECK (signal_type IN (
    'litigation','regulatory_action','media_sentiment','incident'
  )),
  source            TEXT,
  title             TEXT,
  summary           TEXT,
  severity          TEXT CHECK (severity IN ('low','medium','high','critical')),
  url               TEXT,
  discovered_at     TIMESTAMPTZ,
  data              JSONB DEFAULT '{}'
);

```

2.2 Vector Embeddings Table

```

-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- document_embeddings: for semantic search across governance docs
CREATE TABLE document_embeddings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

document_id      UUID REFERENCES documents(id) ON DELETE CASCADE,
chunk_index      INTEGER NOT NULL,
chunk_text       TEXT NOT NULL,
embedding        vector(1536),           -- OpenAI ada-002 or similar
metadata        JSONB DEFAULT '{}',
created_at       TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON document_embeddings
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

3. API Design (FastAPI)

3.1 API Route Structure

```

app/
├── main.py           # FastAPI app, CORS, middleware
├── config.py         # Settings via pydantic-settings
├── database.py       # SQLAlchemy async engine + session
├── models/           # SQLAlchemy ORM models
│   ├── organization.py
│   ├── assessment.py
│   ├── document.py
│   ├── extraction.py
│   └── score.py
├── schemas/          # Pydantic request/response schemas
├── routers/          # API route handlers
│   ├── assessments.py # CRUD + trigger processing
│   ├── documents.py   # Upload, classify, list
│   ├── extractions.py # View extraction results
│   ├── scores.py      # View/configure scoring
│   └── reports.py     # Generate/download PDF reports
├── services/         # Business logic layer
│   ├── document_processor.py
│   ├── governance_extractor.py
│   ├── external_signals.py
│   ├── scoring_engine.py
│   └── report_generator.py
├── llm/              # Claude API integration
│   ├── client.py      # Anthropic SDK wrapper
│   ├── prompts/       # Extraction prompt templates
│   └── schemas.py     # Tool use schemas for structured output
└── workers/          # Background task processing
    ├── pipeline.py    # Orchestrates the full assessment pipeline
    └── tasks.py       # Individual async tasks

```

3.2 Key API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/assessments	Create new assessment for an organization
POST	/api/v1/assessments/{id}/documents	Upload governance documents (multipart)
POST	/api/v1/assessments/{id}/process	Trigger the full assessment pipeline
GET	/api/v1/assessments/{id}	Get assessment status + results
GET	/api/v1/assessments/{id}/extractions	View all extractions with evidence
GET	/api/v1/assessments/{id}/scores	View dimension scores + flags
GET	/api/v1/assessments/{id}/report	Download PDF risk report
PUT	/api/v1/assessments/{id}/config	Update carrier-specific scoring weights
GET	/api/v1/organizations/{id}/history	View assessment history for an org
GET	/api/v1/signals/search	Search external signals by company/industry

3.3 Assessment Pipeline (Async Processing)

When an underwriter hits POST /process, this async pipeline runs:

```

async def run_assessment_pipeline(assessment_id: str):
    # Step 1: Process all uploaded documents
    docs = await get_documents(assessment_id)
    for doc in docs:
        await process_document(doc)      # Parse, OCR, chunk
        await classify_document(doc)      # Claude classifies doc type
        await generate_embeddings(doc)    # Create vector embeddings

    # Step 2: Extract governance data per dimension
    for dimension in GOVERNANCE_DIMENSIONS:
        relevant_docs = await find_relevant_docs(
            assessment_id, dimension      # Semantic search
        )
        extraction = await extract_governance_data(
            dimension, relevant_docs      # Claude structured extraction
        )
        await save_extraction(extraction)

    # Step 3: Gather external signals
    org = await get_organization(assessment_id)
    signals = await gather_external_signals(org)
    await save_signals(assessment_id, signals)

```

```
# Step 4: Score all dimensions
extractions = await get_all_extractions(assessment_id)
config = await get_scoring_config(assessment_id)
scores = scoring_engine.calculate(extractions, signals, config)
await save_scores(assessment_id, scores)

# Step 5: Generate report
report_url = await generate_pdf_report(assessment_id)
await update_assessment(assessment_id, status='complete',
                        report_url=report_url)
```

4. Claude API Integration

4.1 Document Classification Prompt

The first Claude call classifies each uploaded document into one of our governance taxonomy types:

```
CLASSIFICATION_SYSTEM_PROMPT = """
You are an AI governance document classifier for insurance
underwriting. Classify the provided document into exactly one
of these categories:

- model_card: Describes a specific AI/ML model's purpose,
  performance, limitations, and intended use
- model_inventory: Lists all AI/ML systems deployed by the org
- bias_audit: Documents bias testing, fairness metrics, or
  disparate impact analysis
- hitl_policy: Human-in-the-loop policies for AI decisions
- incident_response: AI failure/incident response procedures
- data_governance: Data handling, privacy, retention policies
- dpia: Data Protection Impact Assessment
- deployment_arch: System architecture, infrastructure docs
- third_party_risk: Vendor AI risk assessments
- regulatory_compliance: Compliance frameworks (EU AI Act, etc.)
- other: Relevant but doesn't fit above categories
- unknown: Not related to AI governance

Respond using the classify_document tool.
"""
```

4.2 Governance Extraction with Structured Output

Each governance dimension has a dedicated extraction prompt that uses Claude's `tool_use` feature to produce typed, structured output. Here's the `human_oversight` dimension:


```

HUMAN_OVERSIGHT_TOOL = {
    "name": "extract_human_oversight",
    "description": "Extract human oversight governance data",
    "input_schema": {
        "type": "object",
        "required": ["has_hitl_policy", "oversight_findings"],
        "properties": {
            "has_hitl_policy": {
                "type": "boolean",
                "description": "Does the org have a documented HITL policy?"
            },
            "consequential_decisions_covered": {
                "type": "array",
                "items": {"type": "string"},
                "description": "List of decision types with human oversight"
            },
            "escalation_path_documented": {"type": "boolean"},
            "override_authority_defined": {"type": "boolean"},
            "review_frequency": {"type": "string"},
            "oversight_findings": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "finding": {"type": "string"},
                        "severity": {"enum": ["info", "warning", "critical"]},
                        "evidence_page": {"type": "integer"},
                        "evidence_quote": {"type": "string"}
                    }
                }
            },
            "confidence": {"type": "number", "minimum": 0, "maximum": 1}
        }
    }
}

```

4.3 Claude API Call Pattern

```

import anthropic

client = anthropic.Anthropic() # uses ANTHROPIC_API_KEY env var

async def extract_dimension(dimension: str, doc_chunks: list[str]):
    prompt_template = DIMENSION_PROMPTS[dimension]
    tool_schema = DIMENSION_TOOLS[dimension]

    response = client.messages.create(
        model="claude-sonnet-4-20250514",

```

```

max_tokens=4096,
system=prompt_template['system'],
tools=[tool_schema],
tool_choice={"type": "tool", "name": tool_schema['name']},
messages=[{
    "role": "user",
    "content": f"""Analyze the following governance
documentation excerpts and extract structured data
about {dimension}."""

    Documents:\n{'\n---\n'.join(doc_chunks)}"""
}]
)

# Extract structured data from tool use response
tool_use = next(
    block for block in response.content
    if block.type == "tool_use"
)
return tool_use.input # Already typed/validated by Claude

```

WHY CLAUDE SONNET FOR EXTRACTION, OPUS FOR CLASSIFICATION

Use Sonnet for the high-volume extraction work (8 dimensions x N documents = many calls). It's fast, cheap, and accurate for structured extraction. Reserve Opus for the initial document classification and the final report narrative generation where nuanced reasoning matters most. This keeps API costs under \$2–\$5 per full assessment.

5. Frontend Architecture

5.1 React App Structure

```

frontend/
├── src/
│   ├── app/                                # Next.js App Router (or Vite + React Router)
│   │   ├── layout.tsx
│   │   ├── page.tsx                        # Dashboard: recent assessments
│   │   └── assessments/
│   │       ├── new/page.tsx               # Create assessment + upload docs
│   │       └── [id]/
│   │           ├── page.tsx               # Assessment detail + score card
│   │           ├── extractions/           # Drill into dimension extractions
│   │           └── report/                # View/download PDF report
│   └── settings/                          # Carrier scoring config
├── components/
└── ScoreCard.tsx                         # Visual score summary (radar chart)

```

			DimensionBadge.tsx	# Red/amber/green per dimension
			DocumentUploader.tsx	# Drag-and-drop multi-file upload
			EvidenceTrail.tsx	# Shows source doc + page for each finding
			ProcessingStatus.tsx	# Real-time pipeline progress
			FlagsList.tsx	# Risk flags with severity indicators
			hooks/	
			useAssessment.ts	# TanStack Query hook for assessment data
			useDocumentUpload.ts	# Upload with progress tracking
			lib/	
			api.ts	# Axios/fetch wrapper for backend
			types.ts	# TypeScript types matching Pydantic schemas

5.2 Key UI Screens

Screen	Components	Data Source
Dashboard	Assessment list, status badges, quick stats	GET /assessments (paginated)
New Assessment	Org form, DocumentUploader, doc type preview	POST /assessments + POST /documents
Assessment Detail	ScoreCard (radar chart), DimensionBadges, FlagsList	GET /assessments/{id} + /scores
Extraction Drill-Down	EvidenceTrail, source document viewer, confidence badges	GET /extractions (filtered by dimension)
Report View	Embedded PDF viewer + download button	GET /assessments/{id}/report
Settings	Scoring weight sliders per dimension, carrier config	PUT /assessments/{id}/config

6. Deployment Pipeline

6.1 Infrastructure (AWS)

Service	AWS Resource	Config
Backend API	ECS Fargate	1 service, 2 tasks (auto-scaling), 512MB/0.25 vCPU each
Background Workers	ECS Fargate	Separate service for async pipeline processing
Database	RDS PostgreSQL 16	db.t4g.medium (2 vCPU, 4GB); pgvector enabled
Document Storage	S3	Private bucket; presigned URLs for upload/download

Frontend	S3 + CloudFront	Static site deployment; edge-cached globally
Secrets	AWS Secrets Manager	API keys (Anthropic, etc); DB credentials
Queue	SQS (or Redis via ElastiCache)	Pipeline task queue for background workers
DNS + SSL	Route 53 + ACM	Custom domain; auto-renewed SSL certificate

6.2 CI/CD with GitHub Actions

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/ --cov=app --cov-fail-under=80

  deploy-backend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
      - uses: aws-actions/amazon-ecr-login@v2
      - run: |
          docker build -t $ECR_REPO:$GITHUB_SHA .
          docker push $ECR_REPO:$GITHUB_SHA
      - run: aws ecs update-service --force-new-deployment

  deploy-frontend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: npm ci && npm run build
      - run: aws s3 sync dist/ s3://$BUCKET --delete
      - run: aws cloudfront create-invalidation --paths "/*"
```

6.3 Local Development

```
# docker-compose.yml for local dev
services:
```

```

db:
  image: pgvector/pgvector:pg16
  ports: ["5432:5432"]
  environment:
    POSTGRES_DB: ailra
    POSTGRES_PASSWORD: localdev

api:
  build: ./backend
  ports: ["8000:8000"]
  volumes: [".:/backend:/app"]
  environment:
    DATABASE_URL: postgresql+asyncpg://postgres:localdev@db/ailra
    ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
  depends_on: [db]

frontend:
  build: ./frontend
  ports: ["3000:3000"]
  volumes: [".:/frontend/src:/app/src"]

```

7. Phased Development with Claude Code

This is the section that matters most. Below are six development phases, each with the exact Claude Code commands, sub-agent configurations, and CLAUDE.md instructions you'll use to build each component. The pattern is: set up project conventions first, then let Claude Code build iteratively with sub-agents handling focused tasks.

CLAUDE CODE PROJECT SETUP — DO THIS FIRST

Before writing any code, create a CLAUDE.md file in your repo root. This is Claude Code's persistent memory. It should contain your tech stack decisions, coding conventions, file structure, and project-specific rules. Claude Code reads this file at the start of every session.

CLAUDE.md Template

```

# CLAUDE.md

## Project: AI Liability Risk Assessment Agent (AILRA)

## Tech Stack
- Backend: Python 3.12, FastAPI, SQLAlchemy 2.0 (async), Alembic
- Database: PostgreSQL 16 with pgvector
- Frontend: React 18, TypeScript, Tailwind CSS, shadcn/ui

```

- LLM: Anthropic Claude API (Sonnet for extraction, Opus for classification)
- Testing: pytest + pytest-asyncio (backend), Vitest (frontend)

Conventions

- Use async/await everywhere in the backend
- All API responses use Pydantic schemas - never return raw dicts
- Database models go in app/models/, schemas in app/schemas/
- Business logic in app/services/ - routers are thin
- Every extraction prompt lives in app/llm/prompts/ as a .py file
- Use dependency injection for DB sessions and API clients
- Type hints required on all function signatures
- Tests mirror source structure: tests/services/test_scoring.py

Commands

- `docker compose up -d db` to start local Postgres
- `uvicorn app.main:app --reload` to run the API
- `pytest tests/ -x` to run tests (stop on first failure)
- `alembic upgrade head` to apply migrations
- `cd frontend && npm run dev` to start the React app

Important Rules

- NEVER hardcode API keys. Use environment variables via pydantic-settings.
- NEVER use print() for logging. Use structlog.
- ALWAYS add Alembic migration when changing models.
- ALWAYS write at least one test per service function.
- Claude API calls MUST use tool_use for structured output.
- Scoring engine MUST be deterministic (no LLM in scoring).

PHASE 1 — Weeks 1-2

Foundation & Document Ingestion

Goal: Set up the project skeleton, database, document upload, and basic PDF parsing. At the end of this phase, an underwriter can upload documents and see them classified by type.

Step 1: Project Scaffolding

Open Claude Code in a fresh repo and give it this prompt:

Create the project structure for AILRA - an AI liability risk assessment tool. Set up:

1. Python backend with FastAPI in /backend
 - Use the project structure from CLAUDE.md
 - Include pyproject.toml with all dependencies
 - Set up SQLAlchemy 2.0 async with Alembic
 - Create the database models for: organizations, assessments,

- documents (from the schema in the tech architecture doc)
 - Set up pydantic-settings for config
 - Create a health check endpoint
2. Docker compose for local dev (Postgres with pgvector)
3. React frontend in /frontend
- Vite + React 18 + TypeScript + Tailwind + shadcn/ui
 - Basic layout with sidebar navigation
 - Placeholder pages for Dashboard, New Assessment, Settings

Think hard about the project structure before starting.

Step 2: Document Upload & Processing

Use a sub-agent for the document processing service:

```
# .claude/agents/document-processor.md
---
name: document-processor
description: Builds document ingestion pipeline
tools: Read, Write, Edit, Bash, Grep
---
```

You are a document processing engineer. Build the document upload and processing pipeline for insurance governance documents.

Requirements:

- S3 upload via presigned URLs (use boto3)
- PDF text extraction using unstructured.io
- Text chunking with overlap (1000 chars, 200 overlap)
- Store raw text and chunks in the documents table
- Handle: PDF, DOCX, images (OCR via unstructured)
- Async processing via background tasks
- Write tests using pytest-asyncio with a test PDF fixture

Then in your main Claude Code session:

```
Use the document-processor sub-agent to build the full document
upload and processing pipeline. It should create:
- POST /api/v1/assessments/{id}/documents endpoint
- S3 presigned URL generation
- Background task that processes uploaded documents
- Text extraction service using unstructured
- Tests with a sample PDF fixture
```

Step 3: Document Classification with Claude

Build the document classification service in `app/services/document_classifier.py` that:

1. Takes a document's extracted text (first 4000 chars)
2. Calls Claude Sonnet via the Anthropic SDK
3. Uses `tool_use` with a `classify_document` tool that returns:


```
{doc_type: string, confidence: float, reasoning: string}
```
4. Updates the document record with the classification
5. Write the classification prompt in `app/llm/prompts/classify.py`
6. Add tests with mock Anthropic responses

Use the classification categories from `CLAUDE.md`.

PHASE 2 — Weeks 3–4

Governance Extraction Engine

Goal: Build the Claude-powered extraction engine that pulls structured governance data from each document, organized by the 8 scoring dimensions.

Step 1: Extraction Prompt Engineering

Create a sub-agent specifically for prompt engineering:

```
# .claude/agents/prompt-engineer.md
---
name: prompt-engineer
description: Designs and tests extraction prompts for governance dimensions
tools: Read, Write, Edit, Bash
---
```

You are an expert at designing Claude extraction prompts with `tool_use` for structured output. You understand insurance underwriting and AI governance frameworks (EU AI Act, NIST RMF, ISO 42001).

For each governance dimension, create:

1. A system prompt explaining what to extract
2. A `tool_use` schema with typed fields and descriptions
3. Evidence linking (page numbers, quotes) in every extraction
4. A confidence score for the overall extraction

Each prompt must be insurance-underwriting-aware: findings should be framed as risk indicators, not generic observations.

Then prompt Claude Code:

Use the prompt-engineer sub-agent to create extraction prompts for ALL 8 governance dimensions:

1. `model_inventory` - Extract: system count, risk classifications, deployment environments, model types
2. `human_oversight` - Extract: HITL policies, escalation paths, override authority, review frequency
3. `bias_fairness` - Extract: testing methodology, protected classes analyzed, remediation records, testing frequency
4. `data_governance` - Extract: data provenance, consent mechanisms, retention policies, cross-border flows
5. `incident_response` - Extract: IR plan existence, notification procedures, post-incident review, SLA definitions
6. `monitoring_drift` - Extract: monitoring frequency, drift detection methods, retraining triggers, alerting
7. `regulatory_compliance` - Extract: frameworks claimed, gap analysis results, audit history, certifications
8. `third_party_risk` - Extract: vendor AI inventory, contractual protections, assessment frequency, supply chain depth

Put each in `app/llm/prompts/{dimension}.py` with the system prompt and `tool_use` schema. Think deeply about what an insurance underwriter needs to assess risk.

Step 2: Extraction Service

Build `app/services/governance_extractor.py` that:

1. For a given assessment, identifies which documents are relevant to each dimension (use document classification)
2. Chunks relevant documents and selects the best chunks via embedding similarity (pgvector)
3. Calls Claude with the dimension-specific prompt + `tool_use`
4. Stores the structured extraction in the `extractions` table
5. Preserves the full evidence chain: `document_id`, page numbers, source quotes
6. Handles errors gracefully (LLM timeouts, malformed output)
7. Logs token usage for cost tracking

Use Claude Sonnet for extractions (fast + cheap).
Process dimensions in parallel using `asyncio.gather()`.

PHASE 3 — Weeks 5–6

Scoring Engine & Report Generation

Goal: Build the deterministic scoring engine and PDF report generator. At the end of this phase, you have a working end-to-end pipeline: upload docs → extract → score → PDF report.

Step 1: Scoring Engine

Build `app/services/scoring_engine.py` - a DETERMINISTIC, rules-based scoring engine (NO LLM calls in scoring).

Requirements:

1. Each dimension scored 0-100 based on extraction data
2. Scoring rules are explicit and configurable per carrier:
 - Default weights from the governance taxonomy
 - Carriers can override weights via assessment config
3. For each dimension, define specific scoring criteria:
Example for `human_oversight` (weight: 20%):
 - `has_hitl_policy = True`: +30 points
 - `consequential_decisions_covered >= 3`: +20 points
 - `escalation_path_documented = True`: +20 points
 - `override_authority_defined = True`: +15 points
 - `review_frequency` is 'quarterly' or better: +15 points
4. Generate flags (red/amber/green) per dimension
5. Calculate overall weighted score
6. Assign risk tier: low (75-100), medium (50-74), high (25-49), critical (0-24)
7. Write extensive tests with various extraction scenarios

The scoring MUST be explainable - every point must trace back to a specific extraction finding.

Step 2: PDF Report Generator

Build `app/services/report_generator.py` using WeasyPrint:

1. Create an HTML template in `app/templates/report.html`
2. Report sections:
 - Cover page: org name, date, assessor, overall score
 - Executive summary: 3-sentence risk narrative (use Claude Opus to generate this from the scored data)
 - Score card: radar chart (use matplotlib to render SVG), overall score, risk tier badge
 - Dimension details: for each of 8 dimensions, show score, flags, key findings, evidence citations
 - External signals: litigation/regulatory summary
 - Methodology: explain scoring approach for transparency
 - Appendix: document inventory with classifications
3. Professional styling: use CSS that matches insurance report conventions (conservative colors, clear hierarchy)
4. Upload generated PDF to S3, store URL in assessment record

Use a sub-agent for the HTML/CSS design work:

"Use a sub-agent to design the report HTML template with

```
professional insurance report styling."
```

PHASE 4 — Weeks 7–8

External Signals & Enrichment

Goal: Add external risk signal enrichment — AI litigation data, regulatory actions, and media sentiment that supplements the governance document analysis.

Step 1: Litigation Data Scraper

Build `app/services/external_signals.py` with:

1. PACER API integration for federal AI litigation
 - Search by company name + AI-related keywords
 - Extract: case type, filing date, status, allegations
2. State court scraper for major jurisdictions
(CA, NY, IL, TX, DE - highest AI litigation volume)
3. Regulatory action tracker:
 - FTC enforcement actions mentioning AI
 - EU AI Office public database
 - State AG actions (CO, IL, NY)
4. Media sentiment via news API:
 - Search company name + 'AI' + risk-related terms
 - Use Claude to classify sentiment as positive/neutral/negative
5. Store all signals in `external_signals` table
6. Calculate signal severity based on recency + relevance

Implement as async tasks that run in parallel.

Cache results for 24 hours to avoid redundant API calls.

PHASE 5 — Weeks 9–10

Frontend Dashboard

Goal: Build the underwriter-facing React dashboard. Use Claude Code heavily here — UI work is where sub-agents shine.

Step 1: Dashboard Components

Use a frontend sub-agent:

```
# .claude/agents/frontend-builder.md
```

```

---
name: frontend-builder
description: Builds React/TypeScript frontend components
tools: Read, Write, Edit, Bash, Grep
---

```

You are a senior React/TypeScript developer building an insurance underwriting dashboard. Use:

- React 18 with hooks
- TypeScript (strict mode)
- Tailwind CSS for styling
- shadcn/ui for base components
- TanStack Query for API data fetching
- Recharts for data visualization

Design principles:

- Insurance professionals expect clean, conservative UI
- Information density matters - show data, not decoration
- Every score/flag must link to its evidence source
- Use color coding: green (safe), amber (warning), red (risk)

Use the frontend-builder sub-agent to build these components in sequence:

1. ScoreCard.tsx - Radar chart showing 8 dimension scores, overall score prominently, risk tier badge. Use Recharts RadarChart. Clicking a dimension navigates to detail view.
2. DocumentUploader.tsx - Drag-and-drop zone for PDFs/DOCX. Show upload progress. After upload, show classified doc type with confidence badge. Allow re-classification.
3. ProcessingStatus.tsx - Real-time pipeline progress showing which step is active (uploading -> classifying -> extracting -> scoring -> generating report). Poll GET /assessments/{id} every 3 seconds while status != 'complete'.
4. EvidenceTrail.tsx - For a given finding, show the source document name, page number, and quoted text. Clicking opens a document viewer with the relevant page highlighted.
5. FlagsList.tsx - Sortable/filterable list of all risk flags across all dimensions. Color-coded by severity. Each links to the extraction evidence.

PHASE 6 — Weeks 11–12

Polish, Security & Launch

Goal: Harden security, add auth, write documentation, and prepare for first customer pilot.

Step 1: Security Hardening

Use a sub-agent for security review:

```
"Use the security-reviewer sub-agent to audit the entire codebase for: SQL injection, XSS, auth bypass, secrets in code, insecure file handling, SSRF via document upload, rate limiting, and CORS configuration."
```

Then fix all findings and:

1. Add Supabase Auth with JWT validation middleware
2. Implement Row Level Security in Postgres
3. Add rate limiting (slowapi) on all endpoints
4. Ensure S3 uploads use presigned URLs (no direct access)
5. Add input validation on all file uploads (type, size)
6. Set up Sentry for error tracking
7. Add request logging with structlog

Step 2: Testing & Documentation

1. Use Claude Code agent teams (Writer + Reviewer pattern):
 - Writer agent: generates integration tests for the full assessment pipeline (upload -> classify -> extract -> score -> report)
 - Reviewer agent: reviews the tests for edge cases and missing coverage
2. Generate API documentation:
 - FastAPI auto-generates OpenAPI spec at /docs
 - Add detailed docstrings to all endpoints
 - Create a Postman collection for customer onboarding
3. Create a demo script that runs a full assessment on a sample set of governance documents (include 3-5 sample docs in the repo for testing/demos)

8. Claude Code Workflow Tips

These patterns will maximize your velocity as a two-person team:

8.1 The Explore-Plan-Execute Pattern

For every major feature, follow this three-step pattern:

- **Explore:** "Read the codebase and understand how assessments are currently processed. List all files involved."
- **Plan:** "Think deeply about how to add external signals enrichment. Write a detailed plan before writing any code. Consider error handling, testing, and how it integrates with the existing pipeline."
- **Execute:** "Now implement the plan. Start with the database model and migration, then the service layer, then the API endpoint, then tests."

8.2 Sub-Agent Roster

Create these sub-agents in `.claude/agents/` at the start of the project:

Agent	Purpose	When to Use
document-process or	PDF parsing, text extraction, chunking	Phase 1: building ingestion pipeline
prompt-engineer	Designing Claude extraction prompts + tool schemas	Phase 2: all extraction prompt work
frontend-builder	React/TypeScript/Tailwind components	Phase 5: all frontend work
security-reviewer	Audits code for vulnerabilities	Phase 6: security hardening; also periodically during dev
test-writer	Generates pytest tests from implementation	Every phase: run after completing each service
db-migration	Creates Alembic migrations from model changes	Whenever models change

8.3 Git Worktrees for Parallel Claude Code Sessions

When both founders are working simultaneously, use Git worktrees so each has an isolated copy of the codebase:

```
# Create worktrees for parallel work
git worktree add ../ailra-frontend feature/frontend
git worktree add ../ailra-extraction feature/extraction

# Founder 1 works on extraction engine
cd ../ailra-extraction
claude # Start Claude Code session focused on backend

# Founder 2 works on frontend simultaneously
```

```
cd ../ailra-frontend
claude # Start Claude Code session focused on UI

# Merge when features are complete
git checkout main && git merge feature/frontend
```

8.4 Slash Commands for Repeated Tasks

```
# .claude/commands/new-dimension.md
# Creates everything needed for a new governance dimension
Create a new governance extraction dimension called $ARGUMENTS:
1. Extraction prompt in app/llm/prompts/{dimension}.py
2. Tool use schema with typed fields
3. Scoring rules in app/services/scoring_rules/{dimension}.py
4. Tests in tests/services/test_{dimension}_extraction.py
5. Update the GOVERNANCE_DIMENSIONS constant

# Usage: /new-dimension third_party_risk
```

```
# .claude/commands/full-test.md
# Runs the full test suite and reports results
Run the complete test suite:
1. cd backend && pytest tests/ -v --tb=short
2. cd frontend && npm run test
3. Summarize: total passed, failed, coverage %
4. If any failures, diagnose the root cause
```

8.5 Key Prompting Patterns

For complex features: "Think hard about this before writing code" or "ultrathink" triggers extended reasoning in Claude Code. Use this for architecture decisions, not for simple CRUD.

For code reviews: Spin up a fresh Claude Code session to review code. A fresh context is unbiased: "Review the scoring engine in app/services/scoring_engine.py. Check for: edge cases, missing error handling, off-by-one errors in scoring, and whether the scoring is truly deterministic."

For parallel batch work: "Process all 8 dimension prompt files: for each one, add input validation and error handling. Use sub-agents to parallelize."

For debugging: Paste the full error traceback and say: "This error occurs when processing a document with no text content. Trace through the code path and identify where the null check is missing. Fix it and add a test for this edge case."

9. Infrastructure & API Cost Estimates

Item	Monthly Cost (MVP)	Monthly Cost (10 Customers)	Notes
AWS ECS Fargate (backend)	\$15–\$30	\$50–\$100	0.25 vCPU, 512MB per task; auto-scales
AWS RDS PostgreSQL	\$30–\$50	\$50–\$100	db.t4g.micro for dev; db.t4g.medium for prod
AWS S3 (document storage)	\$1–\$5	\$5–\$20	~\$0.023/GB; governance docs are small
AWS CloudFront	\$0–\$5	\$5–\$10	Free tier covers most usage
Claude API (Sonnet)	\$50–\$150	\$200–\$600	~\$0.50–\$2 per full assessment (8 dimensions)
Claude API (Opus)	\$10–\$30	\$30–\$100	Classification + report narrative only
Unstructured.io (self-hosted)	\$0	\$0	Open-source; runs in your Docker stack
GitHub Actions CI/CD	\$0	\$0	Free for private repos (2,000 min/month)
Supabase Auth (free tier)	\$0	\$0–\$25	Free up to 50K monthly active users
Sentry + PostHog	\$0	\$0–\$50	Free tiers are generous for MVP
Domain + DNS	\$15	\$15	Route 53 hosted zone + domain registration
TOTAL	\$120–\$285	\$355–\$1,020	Well under \$1K/month at MVP stage

COST PER ASSESSMENT

At 8 extraction calls (Sonnet @ ~\$0.05 each) + 1 classification call (Sonnet @ ~\$0.03) + 1 report narrative (Opus @ ~\$0.15) + external signals calls (~\$0.10), each full assessment costs approximately \$0.75–\$2.00 in API fees. At a \$1,500 price point per assessment, that's a 99%+ gross margin on the AI costs alone.

10. Development Milestones & Checkpoints

Week	Milestone	Definition of Done	Demo-able Output
2	Foundation complete	Upload PDFs, see them classified by type in the UI	Show an underwriter uploading 5 docs and seeing instant classification
4	Extraction engine works	All 8 dimensions extract structured data from real governance docs	Show extraction results with evidence links for a sample company
6	End-to-end pipeline	Full flow: upload → classify → extract → score → PDF report	Hand a complete risk report to a pilot MGA underwriter
8	External signals live	Litigation + regulatory data enriches every assessment	Show how a company with active AI lawsuits scores differently
10	Dashboard polished	Underwriter can self-serve: create assessment, upload, view results	Full product demo to 3+ prospective MGA customers
12	Production launch	Auth, security, monitoring, docs complete; first paying customer	Live assessment for a real applicant with a paying MGA

End of Technical Architecture